

calendar_backend V1 Engineering Design Plan

Python service-layer backend for task planning, scheduling, calendar assignment, and free-time allocation

Version: V1 design plan - updated

Project/library: calendar_backend

Date: June 5, 2026

Source of truth: This updated design plan is derived from the locked 1A architectural decisions and the follow-up feedback that clarified deletion, repetition refresh, subtype tables, system-owned constraints, package descriptions, and implementation roadmap numbering. Cursor implementation chunking remains intentionally deferred; this document defines the target architecture and behavior.

Executive summary

calendar_backend is a Python library/service-layer backend for representing a hierarchical plan tree, resolving concrete tasks, assigning tasks into a calendar, and optionally filling remaining time with free-time activities. V1 prioritizes correctness, readability, deterministic behavior, and explicit domain invariants over UI polish or performance tuning. The architecture uses SQLAlchemy over SQLite with Alembic migrations, service-owned mutations, frozen dataclass DTOs, and solver/planner interfaces that isolate optional scheduling dependencies.

Topic	Decision
Primary deliverable	A Python service-layer backend named calendar_backend.
Persistence	SQLite via SQLAlchemy ORM and Alembic migrations, with Postgres-friendly schema choices.
Scheduling approach	TaskAssignmentService uses an AssignmentSolver interface with OR-Tools CP-SAT as the preferred exact solver and deterministic heuristic fallback.
Time model	Timezone-aware UTC datetimes for persisted timestamps; integer minutes for durations, chunk sizes, and granularity.
Mutation model	All persistence-changing behavior goes through services inside atomic transactions.
Repetition model	Repetitions generate materialized clone subtrees with shifted local constraint rows. Detached clones are not overwritten by template refresh, but ordinary deletion still cascades through descendants.
V1 orchestration	Manual OrchestrationService.refresh_schedule() composes resolve tasks, task assignment, and free-time assignment.

Contents

- 1. Context and goals
- 2. V1 non-goals
- 3. Architecture overview
- 4. Package layout
- 5. Domain model
- 6. Data model
- 7. Service layer
- 8. DTOs and result conventions
- 9. Algorithms
- 10. Persistence and transactions

- 11. Settings and time handling
- 12. Error and warning taxonomy
- 13. Testing strategy
- 14. Implementation roadmap
- Appendix A. Locked architectural decisions from 1A

1. Context and goals

The application models a rooted plan tree under a single master goal. Users define goals, repetitions, and tasks with local time constraints. The backend resolves these definitions into concrete active tasks, assigns valid incomplete tasks across the full master horizon, and records the active calendar task layer. Full refresh can also assign future free time after task assignment succeeds.

1.1 Goals

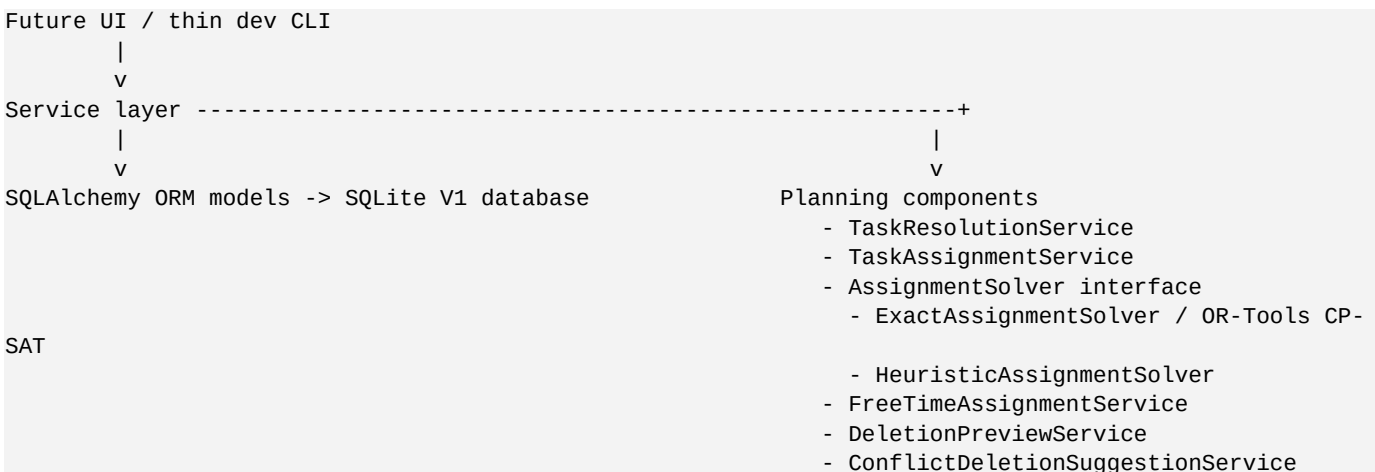
- Represent plans as true subclasses: BasePlan, GoalPlan, RepetitionPlan, and TaskPlan.
- Keep the active plan graph a rooted tree under a single master plan, not a DAG.
- Ensure every active non-master plan is reachable from master; V1 does not support orphan plans.
- Provide service-only mutation with clear transaction boundaries and structured ServiceResult outputs.
- Resolve all tasks into valid/invalid and complete/incomplete buckets with derived scheduling metadata.
- Assign valid incomplete tasks across the full current master horizon while respecting hard feasibility rules.
- Support exact solver-backed assignment with deterministic heuristic fallback that never violates hard constraints.
- Persist active calendar entries and run metadata without implementing undo, audit history, or soft delete in V1.
- Provide a clean backend library foundation that can later be exposed through an HTTP API and mobile frontend.

2. V1 non-goals

- No production HTTP API or mobile frontend implementation in V1.
- No OS-level notification scheduling. Backend-generated notification candidates are deferred.
- No sync layer, multi-device conflict resolution, or external calendar integration.
- No recurring availability constraints; all V1 time constraints are explicit windows.
- No undo system, audit/event log, soft delete, historical active-calendar snapshots, or orphan-plan lifecycle.
- No automatic full orchestration after every plan edit or task completion.
- No mutable plan type, plan-type conversion, or DAG relationships.
- No sub-minute scheduling or automatic rounding of invalid time values.
- No proof that V1 conflict sets are mathematically minimal.

3. Architecture overview

The system is layered around persistence records, domain DTOs, services, and isolated planning/scheduling components. ORM models are persistence records; domain behavior lives in services. The diagram is intentionally PDF-safe text rather than Mermaid.



3.1 Main refresh flow

Caller

```
-> OrchestrationService.refresh_schedule()
    -> TaskResolutionService.resolve_tasks(run_started_at)
        -> refresh master horizon
        -> refresh repetitions and materialized shifted constraints
        -> read current master tree
    <- ServiceResult[ResolveTasksResult]
    -> TaskAssignmentService.assign_tasks(result, run_started_at)
        -> on success, atomically replace future TASK entries
        -> on failure, leave active TASK entries unchanged and persist failed run summary
    <- ServiceResult[AssignmentResult]
    -> if task assignment succeeded:
        FreeTimeAssignmentService.assign_free_time(run_started_at)
        -> replace or clear future FREE_TIME entries
    <- ServiceResult[RefreshScheduleResult]
```

4. Package layout

The package layout is layered. Each path has an ownership boundary so Cursor is less likely to create speculative abstractions or put behavior in the wrong layer.

Path	Description	Ownership boundary	Does not belong
calendar_backend/	Importable backend library root.	Owns reusable application code.	Do not put dev-only scripts here unless importable and tested.
calendar_backend/db/	Database infrastructure package.	Owns metadata base, session factories, and migrations.	No domain scheduling algorithms.
calendar_backend/db/base.py	SQLAlchemy Declarative Base and metadata conventions.	Defines ORM base objects.	No business logic.
calendar_backend/db/session.py	Session maker, transaction helpers, engine creation.	Owns persistence setup and transaction entry points.	No service-specific rules.
calendar_backend/db/migrations/	Alembic environment and revisions.	Owns schema migration history.	No application services.
calendar_backend/models/	SQLAlchemy ORM persistence records.	Owns table mappings and relationships.	No public mutation behavior.
calendar_backend/models/plans.py	Base Plan plus one-to-one subtype detail tables for task, goal, repetition.	Owns plan identity and subtype records.	No task resolution or scheduling behavior.
calendar_backend/models/constraints.py	TimeConstraintGroup and TimeWindow rows.	Owns user and system-owned constraint persistence.	No dynamic constraint intersection algorithms.
calendar_backend/models/repetitions.py	RepetitionInstance and repetition lineage records.	Owns instance metadata and root clone links.	No template refresh algorithm.
calendar_backend/models/calendar.py	CalendarEntry rows for TASK and FREE_TIME entries.	Owns active calendar persistence.	No assignment solver code.
calendar_backend/models/free_time.py	FreeTimeActivity and prerequisite rows.	Owns free-time persistence records.	No free-time assignment algorithm.
calendar_backend/models/settings.py	AppSettings row.	Owns persisted app-wide settings.	No direct update logic outside service.
calendar_backend/models/runs.py	CalendarRun and ActiveCalendarState rows.	Owns run summaries and active refresh failure state.	No orchestration logic.
calendar_backend/domain/	Pure domain value types, enums, errors, DTO/result helpers.	Owns non-persistent types.	No database sessions.
calendar_backend/domain/ids.py	NewType wrappers over UUID such as PlanID.	Owns stable ID types.	No random ID generation policy beyond helpers.
calendar_backend/domain/enums.py	Serializable enum-style codes.	Owns PlanKind, entry types, statuses, constraint kinds, etc.	No service behavior.
calendar_backend/domain/errors.py	Exception classes and service message codes.	Owns error taxonomy.	No persistence.
calendar_backend/domain/time.py	Time window dataclasses, minute alignment helpers, clock protocol.	Owns time-domain primitives.	No direct datetime.now() in services.
calendar_backend/domain/results.py	ServiceResult and common DTO result helpers.	Owns result conventions.	No domain mutation.

Updated from locked 1A architectural decisions and subsequent feedback

calendar_backend/services/	Public service layer.	Owns all persistence-changing behavior and transactions.	No heavy optional solver dependencies.
calendar_backend/services/ plan_tree.py	Create/move/rename/delete/ preview tree operations.	Owns tree mutations and deletion cascade entry points.	No assignment solving.
calendar_backend/services/ master_plan.py	Ensure the unique master plan exists.	Owns master creation/retrieval.	No scheduling.
calendar_backend/services/ master_horizon.py	Maintain the rolling master horizon.	Owns system master horizon constraint updates.	No task assignment.
calendar_backend/services/ task.py	Task completion, reopening, and scheduling-field edits.	Owns TaskPlan mutation and clone detachment.	No calendar assignment.
calendar_backend/services/ repetition.py	Repetition creation, refresh, clone propagation, and system repetition-window constraints.	Owns materialized shifted clone constraints.	No assignment solving.
calendar_backend/services/ task_resolution.py	Resolve current tree into task buckets and precedence constraints.	Owns resolution traversal.	No active calendar mutation.
calendar_backend/services/ task_assignment.py	Coordinate assignment solvers and persist successful TASK entries.	Owns assignment transaction semantics.	No free-time writes.
calendar_backend/services/ free_time_activity.py	Manage free-time activities, fractions, prerequisites.	Owns activity validation.	No task calendar writes.
calendar_backend/services/ free_time_assignment.py	Assign future FREE_TIME entries around current TASK blockers.	Owns free-time calendar layer.	No task assignment writes.
calendar_backend/services/ app_settings.py	Read/update app settings.	Owns settings mutation.	No direct caller updates to settings model.
calendar_backend/services/ time_constraints.py	Edit user-owned local time constraints.	Owns USER constraints only.	Must not edit system-owned constraints.
calendar_backend/services/ invariant_validation.py	Validate master tree and core invariants.	Owns diagnostics for tests/startup/dev mode.	No auto-repair in V1.
calendar_backend/scheduling/	Assignment-specific interfaces and algorithms.	Owns solver and decomposition implementations.	No SQLAlchemy sessions.
calendar_backend/scheduling/ interfaces.py	AssignmentSolver protocol and solver result types.	Owns interface contracts.	No concrete OR-Tools imports.
calendar_backend/scheduling/ exact_cp_sat.py	Optional OR-Tools CP-SAT implementation.	Owns exact solver model.	No service transactions.
calendar_backend/scheduling/ heuristic.py	Deterministic hard-valid fallback solver.	Owns fallback algorithm.	No persistence.
calendar_backend/scheduling/ decomposition.py	Break assignment into independent or mostly independent components.	Owns model-size control helpers.	No database writes.
calendar_backend/scheduling/ conflict_analysis.py	Stage deterministic conflict analysis after solver failure.	Owns AssignmentConflict construction.	No deletion preview persistence.
calendar_backend/deletion/	Deletion preview and conflict deletion suggestions.	Owns deletion impact analysis and ranking.	No task assignment.
calendar_backend/deletion/ preview.py	Pure deletion-impact planning data structures.	Given a candidate operation, describe affected rows.	Does not choose best conflict deletion.
calendar_backend/deletion/ preview_service.py	Service-facing deletion preview.	Owns preview_delete(...) and real deletion impact parity.	No solver conflict ranking.
calendar_backend/deletion/ conflict_suggestions.py	Conflict-specific deletion candidate generation/ranking.	Owns smallest legal deletion suggestion logic.	Does not execute deletion.
calendar_backend/settings/	Static defaults and configuration helpers.	Owns default constants.	No persisted settings mutation.
calendar_backend/settings/ defaults.py	Default app settings values.	Owns initial defaults.	No environment-specific secrets.
calendar_backend/orchestration/	Composed use cases.	Owns refresh_schedule workflow.	No low-level ORM model mutation.
calendar_backend/ orchestration/ refresh_schedule.py	Manual full refresh orchestration.	Calls resolution, task assignment, and free-time assignment.	No duplicate assignment logic.
calendar_backend/tests/	Pytest suite.	Owns invariant, service, algorithm, and integration tests.	No production code.
tools/dev_cli.py	Thin development CLI outside the importable package.	Owns local manual commands for development/testing.	No business logic not callable from services.

Abstraction rule: Only introduce abstractions required by locked decisions or needed to isolate optional dependencies and algorithms. Avoid one-use internal classes and speculative extension points.

5. Domain model

The domain centers on a rooted tree of plans, explicit child chains for goal ordering, bounded repetition expansion, task resolution, task assignment, and free-time assignment.

5.1 Plan hierarchy

Concept	V1 behavior
BasePlan	Shared persistence record fields: plan_id, name, parent_id, clone tracking, timestamps, and local time constraints.
GoalPlan	A plan whose children are organized by GoalChildChain records. Completion is derived from critical chains.
RepetitionPlan	A bounded recurrence definition with an unscheduled template subtree and concrete RepetitionInstance rows.
TaskPlan	A schedulable leaf/subtree task with remaining duration, divisibility, minimum chunk size, and manual completion.
Master plan	A normal GoalPlan row with special invariants: unique, generated ID, name master, parent_id NULL, non-deletable.

5.2 Goal child chains

Goal children are not just unordered parent-child relationships. They are explicit chains. A chain is a precedence sequence; later plans in the chain are resolved but must be scheduled after earlier active predecessors. Critical chains sort before non-critical chains and determine logical goal completion. Deleting one plan inside a chain deletes the whole chain. Ordinary deletion of any plan deletes all descendants, while master can never be deleted.

5.3 Repetition expansion

RepetitionPlan is a bounded generator of concrete instances inside the active planning horizon. The template subtree is unscheduled. Instance 0 is a scheduled clone shifted by $0 * \text{repeat_interval}$. After generation, mode, start_time, and repeat_interval are locked. manual_count mode may increase after generation but may not decrease. date_range mode with unset end_time resolves to the active master horizon end and continues expanding as the master horizon rolls forward.

5.4 Clone detachment and deletion

For repetition instance clones, editing/completing/removing a linked clone detaches that clone and its descendants, not its ancestors. Detachment means the clone is not directly overwritten/refreshed from its template analogue. Detachment does not mean the clone is specially preserved from ordinary deletion. If a detached clone is inside a subtree whose actual parent is deleted, ordinary descendant deletion applies. V1 does not support active orphan plans.

5.5 Time constraints and ownership

Local constraints are stored as AND-of-OR groups: List[List[TimeWindow]]. Empty outer lists mean no local restriction. Empty inner groups are invalid. OR windows are normalized/merged within each group; AND groups are preserved and intersected during resolution. Constraints describe when work may occur, not merely when it may start.

Constraint kind	Owner	Editable through normal time-constraint methods?	Purpose
USER	TimeConstraintService / relevant plan services	Yes	User-authored local availability/work windows.
SYSTEM_REPETITION_WINDOW	RepetitionService	No	System-maintained window bounding a repetition template or generated instance to one repeat interval.

SYSTEM_MASTER_HORIZON	MasterHorizonService	No	System-maintained window bounding all work to the active master horizon.
-----------------------	----------------------	----	--

System-owned repetition-window and master-horizon constraints are visible in user-facing/debug DTOs and in effective constraints, but they are not directly editable through ordinary time-constraint editing methods.

5.6 Calendar model

V1 has one active calendar represented by CalendarEntry rows. Task and free-time entries share the same table and are distinguished by entry_type. TaskAssignmentService owns TASK entries; FreeTimeAssignmentService owns FREE_TIME entries. Active future task entries are replaced atomically only on successful assignment.

6. Data model

This section specifies core SQLAlchemy ORM tables and relationships. Minor indexes may be added during implementation; constraints listed here enforce domain invariants and should be represented in the schema when practical.

Subtype pattern: The schema uses a base Plan table plus one-to-one subtype detail tables. This is similar to joined-table inheritance. TaskPlan, GoalPlan, and RepetitionPlan detail rows use plan_id as both their primary key and a foreign key to Plan because they extend the same logical plan identity rather than creating a separate identity.

Table	Core fields	Notes / invariants
Plan	plan_id UUID PK; plan_kind enum; name str; parent_id UUID nullable FK Plan; is_master bool; cloned_from_id UUID nullable FK Plan; clone_status enum; created_at UTC dt; updated_at UTC dt	Base plan identity table. Only GoalPlan may have is_master true. Non-master active plans require parent_id and must be reachable from master.
TaskPlan fields	plan_id UUID PK/FK Plan; duration_minutes int; divisible bool; minimum_chunk_size_minutes int; user_completed bool; completed_at UTC dt nullable	One-to-one detail row for TaskPlan. duration_minutes is remaining required duration. Non-positive duration is invalid for scheduling.
GoalPlan fields	plan_id UUID PK/FK Plan	One-to-one detail row for GoalPlan. Goal-specific behavior is service-derived from child chains.
RepetitionPlan fields	plan_id UUID PK/FK Plan; repeat_mode enum; start_time UTC dt; repeat_interval_minutes int; manual_count int nullable; end_time UTC dt nullable; template_root_id UUID FK Plan; default_instance_critical bool; generated_at UTC dt nullable	One-to-one detail row for RepetitionPlan. After generation, mode, start_time, and repeat_interval are locked. manual_count/end_time may increase but not decrease according to mode.
GoalChildChain	goal_child_chain_id UUID PK; parent_goal_id UUID FK GoalPlan; is_critical bool; sort_order int; created_at; updated_at	Master child chains must be non-critical.
GoalChildChainItem	goal_child_chain_item_id UUID PK; chain_id UUID FK; child_plan_id UUID FK Plan; position int	Each active non-master child plan appears in exactly one chain item.
TimeConstraintGroup	time_constraint_group_id UUID PK; plan_id UUID FK Plan; group_order int; constraint_kind enum USER/SYSTEM_REPETITION_WINDOW/SYSTEM_MASTER_HORIZON	Represents one AND group containing OR windows. constraint_kind controls ownership and edit permissions.
TimeWindow	time_window_id UUID PK; group_id UUID FK; start_time UTC dt; end_time UTC dt	Half-open [start_time, end_time). start_time < end_time and minute-aligned.
RepetitionInstance	repetition_instance_id UUID PK; repetition_plan_id UUID FK; instance_index int; root_clone_id UUID FK Plan; instance_start_time UTC dt; is_effectively_critical bool	Concrete occurrence; instances do not impose precedence on each other. Shifted constraints are materialized on clone plan rows.

CalendarEntry	calendar_entry_id UUID PK; entry_type enum TASK/FREE_TIME; start_time UTC dt; end_time UTC dt; source_plan_id UUID nullable; source_free_time_activity_id UUID nullable; calendar_run_id UUID nullable; display_label str; created_at; updated_at	Future entries are replaced by owning service. Labels are assignment-time snapshots.
FreeTimeActivity	free_time_activity_id UUID PK; name str; enabled bool; real_fraction decimal/fixed; minimum_block_size_minutes int; created_at; updated_at	Enabled positive fractions must sum to 1.
FreeTimeActivityPrerequisite	prerequisite_id UUID PK; free_time_activity_id UUID FK; source_plan_id UUID FK Plan	Task prerequisite uses user_completed; goal/repetition prerequisite uses logical completion.
CalendarRun	calendar_run_id UUID PK; run_started_at UTC dt; run_finished_at UTC dt nullable; status enum; solver_status enum nullable; conflict_count int; warning_count int; runtime_ms int; created_at	Successful and failed assignment/refresh attempts persist summary metadata.
ActiveCalendarState	singleton_id int PK; active_calendar_run_id UUID nullable; last_refresh_failed bool; last_failure_at UTC dt nullable; last_failure_reason enum nullable; updated_at	Tracks refresh failure state, not plan-tree staleness.
AppSettings	singleton_id int PK; local_timezone str; master_horizon_duration_minutes int; scheduling_granularity_minutes int; exact_solver_time_limit_seconds int; exact_solver_model_size_limit int; heuristic_enabled bool; free_time_week_start_day enum; updated_at	Settings changed only through AppSettingsService.

7. Service layer

Public service methods return `ServiceResult[T]`. Mutation happens only through service methods inside transactions. Internal helpers may return plain values or raise exceptions.

Service	Public methods / signatures	Responsibility
MasterPlanService	ensure_master_exists() -> ServiceResult[GoalPlanDTO]	Create or retrieve the unique master plan.
MasterHorizonService	refresh_master_horizon(run_started_at: datetime) -> ServiceResult[MasterHorizonDTO]	Maintain master horizon [now, now + master_horizon_duration) and SYSTEM_MASTER_HORIZON constraints.
PlanTreeService	create_goal(...), create_task(...), create_repetition(...), move_plan(...), rename_plan(...), delete_plan(...), preview_delete(...)	Service-only plan mutations; validate tree invariants and subtype compatibility. Deletion cascades to descendants.
TimeConstraintService	add_user_constraint(...), update_user_constraint(...), remove_user_constraint(...)	Manage USER constraints only. Reject direct mutation of system-owned constraints.
RepetitionService	refresh_repetitions_inside_horizon(...), update_repetition(...), refresh_template_links(...)	Own repetition expansion, clone refresh, descendant-only detachment rules, and materialized SYSTEM_REPETITION_WINDOW constraints.
TaskService	mark_complete(plan_id: PlanID, at: datetime) -> ServiceResult[TaskDTO]; reopen(plan_id: PlanID) -> ServiceResult[TaskDTO]; update_scheduling_fields(...)	Manual task completion and task scheduling edits. Linked clones detach as required.
TaskResolutionService	resolve_tasks(run_started_at: datetime) -> ServiceResult[ResolveTasksResult]	Refresh horizon and repetitions, then resolve current tree into task buckets and precedence constraints.
TaskAssignmentService	assign_tasks(resolved: ResolveTasksResult, run_started_at: datetime) ->	Assign valid incomplete tasks; atomically replace future TASK entries on success.

	ServiceResult[AssignmentResult]	
ConflictAnalysisService	analyze_assignment_failure(...) -> list[AssignmentConflict]	Stage deterministic conflict analysis after solver/heuristic failure.
DeletionPreviewService	preview_delete(operation: DeletionOperation) -> ServiceResult[DeletionPreview]	Given a candidate deletion operation, compute exactly what would be deleted. Shared by real deletion previews and conflict deletion suggestions.
ConflictDeletionSuggestionService	suggest_for_conflict(conflict: AssignmentConflict) -> list[DeletionCandidate]	Generate candidate legal deletion operations, call DeletionPreviewService, and rank candidates by smallest legal impact and priority tie-breaks.
FreeTimeActivityService	create_activity(...), update_activity(...), enable(...), disable(...), set_prerequisites(...)	Manage free-time activities and validate fraction totals.
FreeTimeAssignmentService	assign_free_time(run_started_at: datetime) -> ServiceResult[FreeTimeAssignmentResult]	Assign future free time using current active TASK entries as blockers.
AppSettingsService	get_settings() -> ServiceResult[AppSettingsDTO]; update_settings(...) -> ServiceResult[AppSettingsDTO]	Only supported path for modifying app settings.
PlanTreeInvariantService	validate_master_tree() -> InvariantValidationResult	Full invariant validation for tests, diagnostics, startup checks, and development mode.
OrchestrationService	refresh_schedule() -> ServiceResult[RefreshScheduleResult]	Manual composed refresh: resolve tasks, assign tasks, then assign free time if task assignment succeeds.

8. DTOs and result conventions

8.1 ServiceResult[T]

```
@dataclass(frozen=True)
class ServiceResult(Generic[T]):
    success: bool
    value: T | None = None
    errors: tuple[ServiceMessage, ...] = ()
    warnings: tuple[ServiceMessage, ...] = ()
    metadata: Mapping[str, Any] = field(default_factory=dict)
```

8.2 Core public DTOs

DTO	Fields
ResolvedTask	plan_id, name, duration_minutes, divisible, minimum_chunk_size_minutes, user_completed, completed_at, effective_time_windows, constraint_sources/debug_effective_constraints, priority_path, criticality_path, parent_path, chain_path, validation_errors
ResolvedPrecedenceConstraint	predecessor_task_id, successor_task_id, source_chain_id, reason
ResolveTasksResult	run_started_at, valid_incomplete, valid_completed, invalid_incomplete, invalid_completed, precedence_constraints, warnings
AssignmentResult	run_started_at, optimization_status, calendar_entries, conflicts, warnings, runtime_ms, solver_diagnostics, calendar_run_id
AssignmentConflict	reason_code, task_ids, window_ids, is_global, is_approximate, explanation, deletion_candidates
CalendarEntryDTO	calendar_entry_id, entry_type, start_time, end_time, source_plan_id, source_free_time_activity_id, display_label, calendar_run_id
FreeTimeAssignmentResult	run_started_at, calendar_entries, unassigned_windows, warnings, runtime_ms
DeletionPreview	root_plan_id, legal_operation, affected_plan_ids, affected_task_ids, affected_calendar_entry_ids, affected_depth_counts_from_master, warnings
DeletionCandidate	legal_operation, deletion_preview, ranking_keys, explanation
RefreshScheduleResult	run_started_at, resolution_result, assignment_result, free_time_result,

Updated from locked 1A architectural decisions and subsequent feedback

Debug/resolution DTOs should expose system-owned constraints as ordinary effective constraints with source metadata so the backend can explain why a task only resolves into a certain interval.

9. Algorithms

9.1 Task resolution

```
def resolve_tasks(run_started_at):
    refresh_master_horizon(run_started_at)
    refresh_repetitions_inside_horizon(run_started_at)
    assert master tree invariants are not corrupted

    traverse master tree in critical-first chain order
    for each TaskPlan:
        collect direct task fields
        compute inherited constraints from task, ancestors, repetition instance, chain/order, and
master
        include USER and system-owned materialized constraints in effective-window computation
        normalize OR windows inside groups
        intersect AND groups during resolution
        classify as valid/invalid and complete/incomplete
        if valid:
            emit effective_time_windows and constraint source metadata
    collect active precedence constraints from chain order
    ignore completed predecessors for active scheduling precedence
    return ResolveTasksResult(four buckets, precedence constraints)
```

9.2 Repetition refresh

```
def refresh_repetitions_inside_horizon(run_started_at):
    for each RepetitionPlan:
        desired_count = count_from_mode(repeat_mode, manual_count, end_time, master_horizon_end)
        create missing instances when desired_count increases
        refuse decreases after generation
        maintain template root SYSTEM_REPETITION_WINDOW [start_time, start_time + repeat_interval)
        for each linked instance clone subtree:
            refresh linked clone fields from template analogues
            materialize shifted USER and SYSTEM_REPETITION_WINDOW constraints on linked clones
            do not overwrite detached clone subtrees
            do not specially preserve detached clones from ordinary deletion cascades
    assert no dangling parent references
```

In manual_count mode, instance_count = manual_count. In date_range mode, resolved_end_time = end_time if set else active_master_horizon_end, and instance_count = floor((resolved_end_time - start_time) / repeat_interval). A date-range repetition with unset end_time expands as the master horizon rolls forward.

9.3 Task assignment

```
def assign_tasks(resolved, run_started_at):
    require resolved.run_started_at == run_started_at
    if resolved.invalid_incomplete:
        return ServiceResult(False, errors=[INVALID_INCOMPLETE_TASKS_BLOCK_ASSIGNMENT])

    tasks = resolved.valid_incomplete inside current master horizon
    previous_entries = load_future_task_entries_as_stability_hints(tasks)
    components = decompose_by independent or mostly independent constraint structure
    persisted_entries = []
```

```

component_statuses = []

for component in components:
    result = exact_solver.solve(component, limits=settings.solver_limits)
    if result.usable:
        choose result
    elif settings.heuristic_enabled:
        result = heuristic_solver.solve(component)
        require hard_constraint_validator(result)
    else:
        result = failure

    if result.failed:
        conflicts = ConflictAnalysisService.analyze(...)
        deletion_candidates = ConflictDeletionSuggestionService.suggest_for_conflicts(conflicts)
        persist failed CalendarRun summary only
        mark ActiveCalendarState.last_refresh_failed = True
        return ServiceResult(False, AssignmentResult(conflicts=conflicts, ...))

    persisted_entries.extend(result.entries)
    component_statuses.append(result.status)

atomically replace future TASK entries with persisted_entries
persist successful CalendarRun summary
clear ActiveCalendarState failure state
return ServiceResult(True, AssignmentResult(weakest_status(component_statuses),
persisted_entries, ...))

```

9.4 Assignment objectives

1. Hard feasibility: duration, windows, no task overlap, minimum chunk size, indivisible exact block, and precedence.
2. Preserve exact previous future windows where possible.
3. Minimize total moved minutes.
4. Minimize number of changed assignments.
5. Improve priority quality.
6. Consolidate free gaps.
7. Schedule earlier as final tie-breaker.

9.5 Conflict analysis

Conflict analysis is staged, deterministic, and useful rather than mathematically minimal. It should distinguish invalid-input precondition failures from valid-task assignment conflicts.

- NO_VALID_WINDOW_FOR_TASK
- INSUFFICIENT_TOTAL_CAPACITY
- PRECEDENCE_IMPOSSIBLE
- MINIMUM_CHUNK_SIZE_IMPOSSIBLE
- TASK_OVERLAP_REQUIRED
- SOLVER_FAILED_TO_FIND_FEASIBLE_ASSIGNMENT

9.6 Deletion preview and conflict deletion suggestions

DeletionPreviewService is the shared utility. Given a candidate deletion operation, it determines exactly what would be deleted, including descendant cascades, chain deletion cascades, critical-parent cascades, calendar entry impacts, and depth/priority metadata. It does not decide which operation is best for a conflict.

ConflictDeletionSuggestionService is conflict-specific. It generates candidate legal deletion operations for a conflict, calls DeletionPreviewService for each, and ranks the resulting candidates. Ranking first protects higher structural levels using

affected_depth_counts_from_master, then protects higher-priority affected items. This is where the smallest legal deletion operation is selected for conflict-resolution suggestions.

9.7 Free-time assignment

```
def assign_free_time(run_started_at):
    load current future TASK entries as blockers
    compute free windows inside active master horizon
    load enabled positive-fraction activities
    evaluate prerequisites using task user_completed or goal/repetition logical completion
    renormalize fractions among unblocked eligible activities without persisting renormalized values
    if no eligible activities:
        clear future FREE_TIME entries where safe
        return success with no entries and warning
    for each local week:
        greedily assign eligible gaps to approach weekly real_fraction targets
        respect each activity.minimum_block_size_minutes
        prefer preserving valid previous future FREE_TIME entries
        prefer fewer and larger contiguous entries
        tie-break earlier
    atomically replace future FREE_TIME entries
    return FreeTimeAssignmentResult
```

10. Persistence and transactions

- Every mutating public service method runs inside an atomic transaction.
- Task assignment failure writes only CalendarRun summary metadata and leaves active TASK entries unchanged.
- Task assignment success atomically replaces future TASK entries only.
- Free-time assignment replaces or clears future FREE_TIME entries only.
- If free-time assignment fails after task assignment succeeds, task assignment remains persisted.
- Hard deletes are used in V1; deletion previews are backend-owned and frontend-confirmed later.
- Deleting a plan deletes all descendants. V1 does not allow active orphan plans or dangling parent references.
- Created_at and updated_at are set by application code via ClockService.
- Alembic migration files are normal development artifacts from the beginning.

11. Settings and time handling

Setting	Type / units	Notes
local_timezone	IANA timezone string	Used for local period logic such as weekly free-time balancing.
master_horizon_duration_minutes	integer minutes	Default should represent the two-year master horizon.
scheduling_granularity_minutes	integer minutes	Locked to 1 in V1.
exact_solver_time_limit_seconds	integer seconds	Limits exact solver runtime.
exact_solver_model_size_limit	integer count/threshold	Prevents massive monolithic models.
heuristic_enabled	bool	Controls fallback behavior.
free_time_week_start_day	enum	Likely Monday; interpreted in local timezone.

12. Error and warning taxonomy

Category	Examples	Handling
Validation failure	INVALID_DURATION, NON_MINUTE_ALIGNED_WINDOW, EMPTY_CONSTRAINT_GROUP, INVALID_TIME_WINDOW,	Return ServiceResult(success=False).

	SYSTEM_CONSTRAINT_DIRECT_EDIT_FORBIDDEN	
Precondition failure	INVALID_INCOMPLETE_TASKS_BLOCK_ASSIGNMENT, RUN_STARTED_AT_MISMATCH, REPETITION_COUNT_DECREASE_AFTER_GENERATION	Return ServiceResult(success=False).
Assignment conflict	NO_VALID_WINDOW_FOR_TASK, PRECEDENCE_IMPOSSIBLE, INSUFFICIENT_TOTAL_CAPACITY	Return AssignmentResult with conflicts; do not replace active task calendar.
Solver warning/status	SOLVER_LIMIT_REACHED, FEASIBLE_NOT_PROVEN_OPTIMAL, HEURISTIC_FEASIBLE	May still return success if a hard-valid assignment is persisted.
Programmer/corruption error	WrongPlanTypeError, corrupted tree invariants, unreachable non-master plan, dangling parent_id	Raise exception, do not hide as user validation.

13. Testing strategy

Testing should be present in the design from the start, while exact Cursor chunk test policy remains deferred. Invariant tests should come before complex scheduling implementation.

Test category	Examples
Invariant tests	Exactly one master; all active non-master plans reachable; only GoalPlan may be master; each child in exactly one chain; deletion never deletes master; no dangling parent references.
Domain validation tests	Zero duration invalid; non-minute-aligned times invalid; empty inner constraint groups invalid; system-owned constraints not directly editable; enabled free-time fractions sum to 1.
Repetition tests	Manual_count cannot decrease after generation; date_range unset end_time expands with master horizon; linked clone constraints are materialized and shifted; detached clones are not overwritten.
Resolution tests	Four buckets; completed predecessors ignored; invalid incomplete tasks block assignment; inherited constraints and constraint sources intersect correctly.
Assignment tests	Hard feasibility validation; previous entries as soft hints; failure leaves active entries unchanged; fallback never violates hard constraints.
Free-time tests	Old free-time entries do not block task assignment; blocked activities renormalize; tiny gaps remain unassigned; failure does not roll back successful task assignment.
Deletion tests	Deletion preview matches real deletion impact; ordinary deletion cascades to descendants; chain deletion cascade; critical chain deletion cascade; conflict deletion suggestions rank candidate previews correctly.
Integration tests	End-to-end refresh_schedule success/failure paths, CalendarRun metadata, ActiveCalendarState failure clearing.

14. Implementation roadmap

This roadmap is intentionally high-level. Cursor implementation chunks should be created later from this design doc.

- Project skeleton, dependency management, SQLAlchemy session setup, Alembic configuration, ClockService, ID NewTypes, enum definitions, and ServiceResult.
- Core ORM model implementation for plans, subtype detail tables, chains, constraints with constraint_kind, settings, run metadata, and calendar entries.
- MasterPlanService, AppSettingsService, MasterHorizonService, TimeConstraintService, and PlanTreeInvariantService.
- PlanTreeService and TaskService with service-only mutation, descendant-cascading deletion, clone detachment, and transaction boundaries.

12. RepetitionService, materialized shifted constraints, repetition expansion, and TaskResolutionService with public DTOs.
13. DeletionPreviewService and ConflictDeletionSuggestionService.
14. TaskAssignmentService interface integration, hard constraint validator, heuristic solver, decomposition scaffolding, and conflict analysis.
15. Optional OR-Tools CP-SAT exact solver module behind AssignmentSolver.
16. FreeTimeActivityService and FreeTimeAssignmentService deterministic greedy planner.
17. OrchestrationService.refresh_schedule and run metadata/ActiveCalendarState behavior.
18. Thin development CLI in tools/, sample-data/reset commands, and broader integration tests.

Appendix A. Locked architectural decisions from 1A

This appendix preserves the locked 1A architectural decisions with the later clarifications made in this revision. The locked 1B decision list is intentionally omitted from this appendix.

Section 1 - Core intent

- Use true subclasses: BasePlan, GoalPlan, RepetitionPlan, and TaskPlan.
- There is no mutable type field and no v1 plan-type conversion.
- Every plan has plan_id: PlanID and name: str.
- Plan names are human-readable and not globally unique.
- Relationships use IDs, not names.
- Every active non-master plan has parent_id: PlanID.
- The active plan structure is a rooted tree under master, not a DAG.
- The backend is database-backed from the start.
- Tasks are manually completed by the user.
- Goals and repetitions are logically completed from critical children or instances.
- Users cannot manually complete goals or repetitions.
- Clone tracking uses cloned_from_id and clone_status.
- For repetition instance clones, editing/completing/removing a linked clone detaches that clone and descendants, not ancestors.
- Template edits propagate recursively through linked clones; detached clones are not directly overwritten by template refresh.
- Detached clones are not specially preserved from ordinary deletion cascades. If their actual parent is deleted, ordinary descendant deletion applies.
- Processing flow is: edit plan tree -> resolve tasks -> assign tasks -> calendar changes.

Section 2 - Goal / child ordering

- Goal children are represented by explicit GoalChildChain records.
- Each active child plan appears in exactly one location in the tree.
- A child chain is a precedence sequence.
- Later items in a chain are resolved but must be scheduled after earlier items. For A -> B, all of A scheduled windows must finish before B starts.
- Critical chains sort before non-critical chains.
- Non-critical chains do not affect goal completion.
- A goal is complete when all critical child chains are complete.
- A goal with zero critical chains is logically complete.
- Deleting one plan inside a chain deletes the whole chain.
- Deleting a critical chain deletes the parent goal and may cascade upward.
- Ordinary deletion of a plan deletes all of its descendants.
- Goal operations happen through services, not direct object mutation.

Section 3 - Repetition

- RepetitionPlan represents bounded recurrence expansion.
- Repetitions expand into concrete RepetitionInstance records within the active planning horizon.
- The core interval field is repeat_interval, not duration.
- Repetition supports manual_count and date_range modes.
- Date-range expansion uses half-open interval logic.
- Before generation, repetition configuration can change freely.
- After generation, mode, start_time, and repeat_interval are locked.
- Manual repetitions and explicit end_time can increase but not decrease after generation.
- In manual_count mode, instance_count = manual_count.
- In date_range mode, resolved_end_time = end_time if set else active master horizon end, and instance_count = floor((resolved_end_time - start_time) / repeat_interval).
- If date_range end_time is unset, the repetition continues expanding as the master horizon rolls forward.
- The repetition template subtree remains editable.
- Linked clones refresh from template edits; detached clones do not.
- The template is unscheduled.
- The template root has a system-owned repetition-window constraint [start_time, start_time + repeat_interval).
- Instance 0 is also a scheduled clone shifted by 0 * repeat_interval.
- Generated repetition clones receive actual shifted TimeConstraintGroup/TimeWindow rows.
- Linked clone descendants receive materialized shifted copies of analogous template descendant constraints.
- System-owned repetition-window constraints are visible in debug/resolution DTOs but editable only by RepetitionService.
- RepetitionPlan stores template_root_id and default_instance_critical.
- Concrete occurrences are stored as RepetitionInstance.
- Repetition completion is logical based on effectively critical instances.
- Repetition instances do not impose scheduling precedence on each other.
- Every clone receives a unique plan_id.
- resolve_tasks() refreshes all repetitions before collecting tasks.

Section 4 - Task

- TaskPlan has duration, divisible, minimum_chunk_size, and user_completed.
- duration means remaining required duration.
- Partial completion is represented by manually reducing duration.
- There is no completed_duration in v1.
- user_completed is the only direct task completion field.
- Zero duration is invalid.
- A task without valid duration is invalid for scheduling.
- resolve_tasks separates valid/invalid and complete/incomplete tasks.
- assign_tasks only schedules valid incomplete tasks.
- assign_tasks cannot run while invalid incomplete tasks are non-empty.
- Completed tasks are informational and not scheduled.
- by is renamed to minimum_chunk_size.
- Divisible task chunks must each be at least minimum_chunk_size.
- Chunks do not need to be exact multiples of minimum_chunk_size.
- Indivisible tasks require one exact-duration window.
- Marking a task complete removes it from future valid incomplete resolved tasks.
- Incomplete tasks under completed goals/repetitions are still scheduled.
- Tasks outside the master horizon are still resolved and treated as scheduling conflicts if unschedulable.
- Task operations go through TaskService and/or PlanTreeService.
- Editing task scheduling fields detaches linked clones.

Section 5 - Time constraints

- BasePlan.time_constraints stores local constraints only.
- Local constraints use AND-of-ORs: List[List[TimeWindow]].
- TimeWindow uses half-open intervals: [start_time, end_time).
- Time constraints are stored as explicit TimeConstraintGroup and TimeWindow records.
- TimeConstraintGroup includes constraint_kind: USER, SYSTEM_REPETITION_WINDOW, or SYSTEM_MASTER_HORIZON.
- Empty outer constraint lists mean no local restriction.
- Empty inner groups are disallowed.
- Concrete windows must satisfy start_time < end_time.
- Persisted datetimes are timezone-aware and stored internally in UTC.
- The user local timezone is stored as an app setting.
- Recurring availability constraints are deferred from v1.
- Constraints mean when work may occur, not merely when work may start.
- Effective task constraints are computed by intersecting task, ancestor, repetition, chain/order, and master constraints.
- Malformed time windows create invalid affected descendant tasks.
- Valid but non-overlapping constraints create scheduling conflicts.
- Time constraint updates happen through services.
- Normal time-constraint editing methods may edit USER constraints only.
- Master has exactly one system-owned master-horizon constraint group with exactly one window.
- Master horizon updates happen through MasterHorizonService.
- The master horizon is a true inherited constraint in v1.
- Previous active calendar entries can be used as future scheduling stability hints.
- Past active calendar entries are not preserved in v1.
- OR-group windows are normalized/merged within a group.
- AND groups are preserved separately and intersected only during resolution.

Section 6 - Master plan

- master is a normal GoalPlan row with special system-root invariants.
- master is created/retrieved through MasterPlanService.ensure_master_exists().
- Exactly one master must exist.
- Master should be database-enforced with a uniqueness rule when possible.
- Master has a generated plan_id, not a magic ID.
- master.name = "master", but the name does not need to be globally unique.
- Users cannot rename master in v1.
- Only a GoalPlan may have is_master = true.
- master.parent_id = NULL.
- Every active non-master plan must be reachable from master.
- V1 does not support active orphan plans or dangling parent references.
- Plan creation is service-only.
- Master uses normal goal completion logic, but this has no v1 behavioral significance.
- Master child chains must always be non-critical.
- Deletion cascades must never delete master.
- Top-level plans live in normal GoalChildChain records under master.
- There is only one active calendar in v1.
- assign_tasks replaces the active calendar task layer atomically.
- Services validate subtype compatibility.
- Mutating service operations should be atomic transactions.
- Full invariant validation exists for tests, diagnostics, startup checks, and possibly development mode.
- Use PlanTreeInvariantService.validate_master_tree().

- V1 does not auto-repair invariant violations.
- Service results use `ServiceResult[T]`.
- Expected user-facing validation failures return structured results.
- Programmer errors and corrupted invariant states raise exceptions.
- Wrong subtype usage raises `WrongPlanTypeError`.

Section 7 - Resolve tasks

- `resolve_tasks` returns four buckets: valid incomplete, valid completed, invalid incomplete, invalid completed.
- `resolve_tasks` returns `ServiceResult[ResolveTasksResult]`.
- Each resolved task is a DTO/value object called `ResolvedTask`.
- `ResolvedTask` includes direct task fields plus derived fields like `effective_time_windows`, constraint source metadata, `priority_path`, `criticality_path`, `parent_path`, `chain_path`, and `validation_errors`.
- `inside_assignment_horizon` is removed in v1.
- Empty `effective_time_windows` means a valid but potentially unschedulable task.
- Invalid task definitions produce invalid records, not scheduling conflicts.
- Chain precedence is represented by separate `ResolvedPrecedenceConstraint` objects.
- Later tasks in a chain are resolved even if earlier tasks are incomplete.
- Completed predecessors are ignored for active scheduling precedence.
- Invalid incomplete tasks globally block assignment.
- `resolve_tasks` does not persist snapshots in v1.
- There is no global tasks variable.
- Public service: `TaskResolutionService.resolve_tasks(run_started_at)`.
- Public resolution refreshes master horizon and repetitions before resolving.
- Internal read-only helper: `_resolve_from_current_tree(run_started_at)`.
- `resolve_tasks` does not modify the active calendar.
- Goal and repetition completion remain logical/derived.
- `priority_path` is computed after critical-first ordering.
- Deletion consequences are not computed during ordinary resolution.

Section 8 - Assignment / scheduling intent

- `TaskAssignmentService.assign_tasks(...)` assigns across the full current master horizon.
- The master horizon remains `[now, now + master_horizon_duration)`.
- All valid incomplete tasks inside the current master horizon participate in assignment.
- `assign_tasks` accepts the full `ResolveTasksResult`.
- `assign_tasks` requires matching `run_started_at`.
- `assign_tasks` refuses to run if invalid incomplete tasks exist.
- Assignment hard feasibility requires full duration, valid windows, no overlaps, minimum chunk compliance, indivisible exact blocks, and precedence satisfaction.
- Precedence is hard feasibility, not a soft objective.
- Task assignment objective is: preserve exact previous future windows where possible; minimize total moved minutes; minimize number of changed assignments; improve priority quality; consolidate free gaps; schedule earlier as final tie-breaker.
- Priority affects successful scheduling, not only conflict ranking.
- Previous future task entries are stability hints only if still valid, incomplete, and schedulable.
- Divisible tasks may have gaps between chunks.
- Divisible assignment softly prefers fewer chunks and closer chunks.
- `TaskAssignmentService` writes only task entries.
- Free-time assignment is separate.
- On successful assignment, future active task calendar entries are atomically replaced.

- On failure, the active task calendar remains unchanged.
- Assignment failure returns structured conflicts.
- Solver-backed exact feasibility/optimization is preferred where reasonable.
- Bounded solver and heuristic fallback are allowed in v1.
- Heuristic fallback may persist only if all hard constraints are satisfied.
- AssignmentResult includes optimization_status.
- ServiceResult.success = True means a valid assignment was persisted.
- ServiceResult.success = False means nothing was written to the task calendar.

Section 9 - Conflict behavior

- Invalid incomplete tasks are precondition failures, not assignment conflicts.
- Invalid task data means scheduling input is unusable.
- Assignment conflicts mean valid tasks cannot jointly be scheduled.
- If the solver cannot prove infeasibility, try deterministic heuristic fallback.
- If heuristic succeeds, persist with warning.
- If heuristic fails, analyze conflicts.
- V1 conflict reasons include: NO_VALID_WINDOW_FOR_TASK, INSUFFICIENT_TOTAL_CAPACITY, PRECEDENCE_IMPOSSIBLE, MINIMUM_CHUNK_SIZE_IMPOSSIBLE, TASK_OVERLAP_REQUIRED, SOLVER_FAILED_TO_FIND_FEASIBLE_ASSIGNMENT.
- Conflicts may be localized or global.
- Use AssignmentConflict DTO.
- Conflict detection uses staged analysis.
- V1 does not require mathematically minimal unschedulable task sets.
- Conflicts should be useful, deterministic, and optionally marked approximate.
- Deletion candidates map to real service calls.
- Add DeletionPreviewService.
- Add ConflictDeletionSuggestionService.
- Real deletion previews and conflict suggestions share deletion preview/impact analysis.
- Conflict deletion suggestions generate candidate legal deletion operations and rank candidate previews.
- Deletion ranking protects higher structural levels first.
- affected_depth_counts_from_master is the primary ranking key.
- Priority protects higher-priority affected items after structural impact.
- Completed and invalid tasks may appear in deletion impact metadata.
- AssignmentResult contains run time, optimization status, calendar entries, conflicts, and warnings.
- Add named ConflictAnalysisService.

Section 10 - Free time

- Free-time assignment runs automatically only after successful task assignment inside full orchestration.
- FreeTimeAssignmentService.assign_free_time(...) is still manually callable.
- Free-time assignment replaces only future free-time entries.
- Free-time assignment is atomic with respect to free-time entries.
- Task and free-time assignment share one CalendarEntry table.
- CalendarEntry.entry_type is TASK | FREE_TIME.
- TaskAssignmentService owns only TASK entries.
- FreeTimeAssignmentService owns only FREE_TIME entries.
- Old free-time entries never constrain task assignment.
- On successful task assignment, only future task entries are replaced.
- During full orchestration, free-time reassignment clears stale free time after task assignment.
- Temporary task/free-time overlap is acceptable before free-time reassignment.
- If free-time assignment fails after task assignment succeeds, task assignment remains persisted.

- On free-time failure, future free-time entries should be cleared first when safely possible.
- Free-time activities are database-backed domain rows.
- Free-time prerequisites use PlanID, not names.
- Free-time activities support enabled: bool.
- Fractions are stored as Decimal or fixed precision.
- Enabled free-time fractions must sum to 1.
- Blocked activities are renormalized during assignment only.
- Renormalized real_fraction values are not persisted.
- If all enabled positive-fraction activities are blocked, free time remains unassigned and future free-time entries are cleared.
- Task prerequisites use user_completed.
- Goal/repetition prerequisites use logical completion.
- Incomplete non-critical descendants under a completed goal/repetition do not block that prerequisite.
- Free-time activities have no availability constraints in v1.
- Free-time balancing is weekly.
- Weekly balancing uses the app timezone, likely Monday 00:00 to Monday 00:00 in America/Chicago.
- Local weekly balance is a soft objective, not hard feasibility.
- divisible_by is renamed to minimum_block_size.
- Free-time blocks must be at least minimum_block_size.
- Free-time blocks do not need to be exact multiples of minimum_block_size.
- Tiny gaps below every eligible activity minimum_block_size remain unassigned.
- No global minimum free-time block size in v1.
- Previous future free-time entries are weak stability hints.
- Free-time assignment failure does not block the whole orchestration.
- Use FreeTimeActivityService and FreeTimeAssignmentService.
- FreeTimeAssignmentService.assign_free_time(run_started_at) returns ServiceResult[FreeTimeAssignmentResult].
- FreeTimeAssignmentService does not require ResolveTasksResult.

Section 11 - Data model / implementation style

- Backend will be written in Python.
- The spec should say: use the latest stable Python version supported by the project dependencies at implementation time, rather than naming a fixed version.
- V1 starts with SQLite.
- Schema and migration style should avoid unnecessary SQLite-only assumptions so Postgres remains feasible later.
- Use SQLAlchemy ORM.
- Use a base Plan table plus one-to-one subtype detail tables for TaskPlan, GoalPlan, and RepetitionPlan.
- ORM models are mostly persistence records.
- Domain behavior is centralized in services, not model methods.
- Internal DTOs and result objects use frozen dataclasses.
- Pydantic is deferred to future API/frontend boundaries.
- IDs use NewType wrappers over UUID, such as PlanID = NewType("PlanID", UUID).
- Mutation happens only through service methods inside transactions.
- No public persistence-changing model methods like task.mark_complete().
- Public service methods return ServiceResult[T].
- Internal helpers may return plain values or raise exceptions.
- Expected user-facing validation failures return structured results.
- Programmer errors and corrupted invariant states raise exceptions.
- Error/warning codes should use enum-style codes internally and be serializable externally.
- Use a layered package layout: db/, domain/, services/, scheduling/, deletion/, tests/.
- Cursor chunks will be created later, after the design doc.
- Use pytest.

- The exact policy for when to write tests will be decided later during Cursor chunking.
- V1 prioritizes correctness/readability over performance.
- Scheduling should be hidden behind an interface.
- The interface should allow multiple implementations, such as exact solver, heuristic, or future alternatives.
- Optional heavy dependencies should remain isolated behind scheduling-specific modules.
- Use an injectable ClockService / clock protocol.
- Avoid direct datetime.now() calls inside core services.
- Persisted times remain UTC; local period logic uses the app timezone.
- Add an OrchestrationService for the full refresh flow.
- Lower-level services remain independently callable.
- Services should support standalone use while allowing orchestration to compose them safely.
- Implement the backend as a Python library/service layer first.
- Web/mobile API comes later.
- Cursor chunk stop rules are not locked here; they will be handled after the engineering design doc, when creating implementation chunks.

Section 12 - Algorithm choice

- Task assignment concretely schedules the full two-year master horizon.
- Scheduling granularity is one minute globally.
- One minute is the minimum time unit across the domain, not only assignment.
- Task durations must be whole-minute durations.
- Minimum chunk sizes must be whole-minute durations.
- Time constraint window starts/ends must be minute-aligned.
- Calendar entry starts/ends must be minute-aligned.
- No seconds/sub-minute scheduling in v1.
- Non-minute-aligned durations, minimum chunk sizes, and time windows are invalid, not rounded.
- Persisted datetimes remain timezone-aware UTC datetimes.
- One-minute granularity is enforced by domain validation, not by a special storage format.
- The first task assignment implementation should be exact solver with heuristic fallback.
- Architecture: TaskAssignmentService -> AssignmentSolver interface -> ExactAssignmentSolver and HeuristicAssignmentSolver.
- The exact solver is attempted first.
- The exact solver may have runtime, node, and/or model-size limits.
- If exact solving cannot produce a usable assignment, deterministic heuristic fallback may produce a schedule only if all hard feasibility rules are satisfied.
- Conflict analysis is staged and deterministic rather than guaranteed minimal.

Section 13 - Persistence / app integration

- Failed CalendarRun records store summary fields only.
- Full structured conflicts are returned in the immediate service result, not persisted as run-history payloads.
- ActiveCalendarState tracks refresh failure state and active run metadata.
- Renames do not mark the calendar stale.
- Calendar entries store denormalized display labels as assignment-time snapshots and do not automatically update on rename.
- Failed assignment attempts are persisted as run metadata.
- Failed assignment attempts do not replace the active calendar and do not preserve old calendar entries as history.
- The dev CLI is intentionally thin and underspecified for now.
- The design doc should say V1 includes a thin development CLI outside the importable library, but command details are deferred to implementation chunking.